

Análisis de datos

Carrera de Especialización en
Inteligencia Artificial
B5 2025



Programa de la materia

1 **Introducción al análisis de datos**
23/10 Flujo de trabajo típico. Aplicaciones. Configuración del entorno. Introducción a las principales bibliotecas. Introducción al EDA.

2 **Análisis exploratorio de datos (EDA)**
30/10 EDA. Estadística robusta. Visualización de variables numéricas y categóricas. Relación entre variables. Data cleaning y data wrangling. Identificación de valores faltantes y outliers.

3 **Taller práctico - parte 1**
6/11 Presentación de grupos de trabajo final. Práctica en salas virtuales: EDA y visualización inicial de un dataset. Puesta en común.

4 **Preprocesamiento y limpieza de datos**
13/11 Flujo de preparación de datos. Prevención de data leakage. Tratamiento de datos faltantes y outliers. Cardinalidad. Codificación. Discretización.

5 **Procesamiento (continuación)**
20/11 Normalización y estandarización. Técnicas avanzadas para el tratamiento de datos faltantes y outliers. Desbalance. Creación de nuevos features.

6 **Reducción de dimensionalidad**
27/11 Métodos de extracción y técnicas para selección de features. **Bonus:** EDA de datos no estructurados.

7 **Taller práctico - parte 2**
4/12 Práctica en salas virtuales: preprocesamiento y feature engineering de un dataset. Puesta en común. **Bonus:** Herramientas para automatizar el EDA.



Martes 9/12 - límite para compartir el material para la exposición final.

8 **Exposición final**
11/12 Exposición de trabajos finales (desdoblada en 2 sesiones, 11/12 y 15/12).

Recap de lo visto en la clase 4



Flujo de preparación de datos. Split.

Dataset original

Estrategia de split

x1	x2	...	target



x1	x2	...	target



x1	x2	...	target

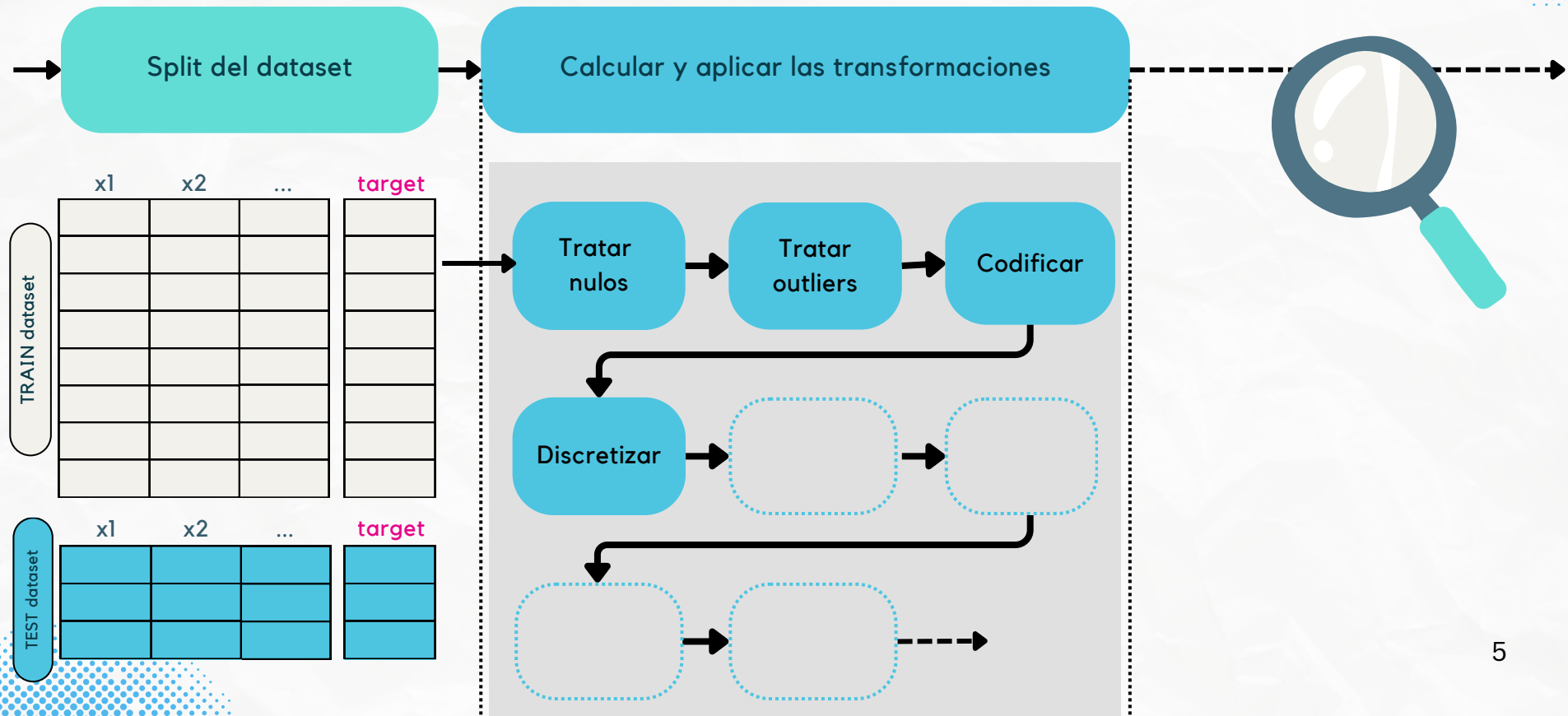
TRAIN
dataset

x1	x2	...	target

TEST
dataset

Evitar target leakage.
Evitar la contaminación train-test.

Flujo de preparación de datos



Normalización y estandarización

- La normalización y la estandarización son dos técnicas para preparar las variables con el objetivo de mejorar el rendimiento de los modelos.
- **Normalización.** Se escalan las variables numéricas para que sus valores estén dentro de un rango específico, generalmente entre 0 y 1.
 - Útil cuando se usa un algoritmo que depende de la distancia (ej. KNN)
- **Estandarización** (o Z-score). Se transforman los datos de modo que tengan una media igual a 0 y una desviación estándar igual a 1. Adecuada cuando los datos ya tienen una distribución normal.
 - Útil para algoritmos que asumen distribución normal (Regresión lineal, SVM)

$$X_{\text{normalizado}} = \frac{X - \min(X)}{\max(X) - \min(X)}$$

$$X_{\text{estandarizado}} = \frac{X - \mu}{\sigma}$$

Actividad



Escalamiento después de codificación

- La decisión de escalar las variables categóricas dependerá del algoritmo de ML a usar y del mecanismo de codificación empleado.
- Excepto algunos casos, la mayoría de las técnicas pueden producir rangos grandes de números y es una buena práctica escalar siempre después de codificar (especialmente si se utilizarán algoritmos basados en distancia).
- En general, no se escalan los features binarios (one-hot encoded) ya que esto no aporta ningún valor (y en algunos casos puede incluso ser contraproducente).
- Tampoco es aconsejable modificar features codificados con cyclic encoding, ya que se "rompe" el efecto cíclico.



Técnicas avanzadas para el tratamiento de datos faltantes



Habíamos visto métodos simples como:

- Eliminación de filas (observaciones) y columnas (variables).
- Imputación por métodos univariados (ejemplo: imputar por la media)

Además, podemos aplicar imputación por métodos *multivariados* (a partir de valores de otras columnas):

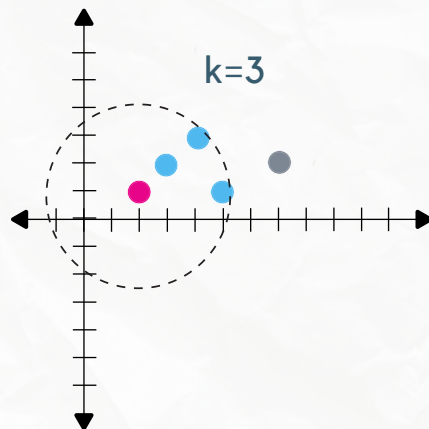
- algoritmos de ML (por ej., regresión, árboles de decisión, KNN).
- modelos de imputación múltiple (ej., MICE)

Métodos multivariados - KNN

KNN (K-Nearest Neighbors):
busca los "k" vecinos que se
parecen más a la fila del dato
faltante. Este se calcula luego a
partir de esos vecinos.

Atención:

- Se recomienda normalizar los datos antes de usar este algoritmo porque es sensible a la escala.
- Puede ser ineficiente en grandes volúmenes de datos.



v1	v2	v3
4	3	2
7	2	2
2	1	NaN
5	1	4
3	2	3

3

KNNImputer

Gallery examples: Imputing missing values before building an estimator
Release Highlights for scikit-learn 0.22



Métodos multivariados - MICE

- MICE (*Multiple Imputation by Chained Equations*) es una técnica iterativa avanzada de imputación múltiple (muchas variables a la vez).
- Es una de las técnicas de imputación multivariada más utilizada.
- Se basa en regresiones lineales aunque puede usar otros modelos.



[Mice imputation \(statsmodels\)](#)

IterativeImputer

Gallery examples: Imputing missing values with variants of
IterativeImputer Imputing missing values before building an estimator



scikit-learn

(En modo experimental. Parecida a MICE, no es igual)

Dataset con nulos

A	B	C

1

Imputación simple

A	B	C
		Media C
Media A		
	Media B	



Ojo! la notación $A \sim B + C$ se llama "fórmula estilo Patsy" y significa que A se predice a partir de B y de C (no es una suma!).

MICE paso a paso

2

Ajustamos un modelo $A \sim B + C$

A	B	C
		Media C
	Media B	

Calculamos los nulos A_i con el modelo y los B_i, C_i

A	B	C
		Media C
Pred A_i	B_i	C_i
	Media B	

3

Ajustamos un modelo $B \sim A + C$

A	B	C
		Media C
Pred A_i		

Calculamos los nulos B_i con el modelo y los A_i, C_i

A	B	C
		Media C
Pred A_i		
A_i	Pred B_i	C_i

4

Ajustamos un modelo $C \sim A + B$

A	B	C
Pred A_i		
	Pred B_i	

Calculamos los nulos C_i con el modelo y los A_i, B_i

A	B	C
A_i	B_i	Pred C_i
Pred A_i		
	Pred B_i	

5

Iteración 1 completa!

A	B	C
		Pred C_i
Pred A_i		
	Pred B_i	

6

Continuar con la próxima vuelta (repetir desde el paso 2 al 5) hasta completar el nro. de iteraciones.

Estrategias avanzadas para tratamiento de faltantes

- Si son MAR con patrones claros y el porcentaje es moderado a alto (20-50%):
 - Imputación con modelos de ML para variables críticas.
 - Algoritmos como *Random Forest Imputer* o KNN-imputer si los patrones son no lineales.
- Si se sospecha MAR con patrones complejos (los faltantes dependen de muchas variables):
 - *Multiple Imputation by Chained Equations* (MICE).
- En el caso de MNAR, se requiere modelar e implementar explícitamente el mecanismo de los faltantes con soluciones *custom*.

Ejemplos prácticos en Jupyter



Técnicas avanzadas para el tratamiento de outliers



Habíamos visto métodos simples como:

- Eliminación de filas (observaciones)
- Imputación por métodos univariados (ejemplo: imputar por la media, *capping*)
- Minimizar el impacto con transformaciones (ejemplo: logaritmo)
- Segmentación

Además, podemos aplicar imputación por métodos *multivariados* (a partir de valores de otras columnas):

- algoritmos de ML (por ej. KNN).

Ejemplos prácticos en Jupyter



Creación de nuevos features



- Se trata de generar nuevas variables a partir de otras existentes.
- El objetivo es captar patrones o relaciones ocultas y mejorar la interpretación.

Ejemplo:

- $IMC = \text{peso} / (\text{altura})^2$
- Reemplazar las variables peso y altura por la métrica IMC, que está más relacionada con la salud que las originales.

Criterios para crear features

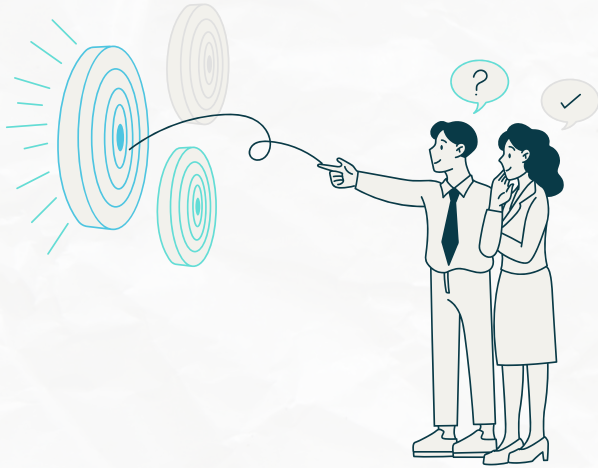


- Deben que ser relevantes para el objetivo.
- Tienen que capturar relaciones importantes o transformaciones de variables guiadas por conocimiento del dominio o análisis.
- Tienen que complementar o superar lo que aportan las variables originales.
- Creados teniendo en cuenta los modelos que los consumirán.
- Hay un balance entre costo computacional e impacto en performance.

Ejemplos de casos de uso:

- Dataset con pocas filas.
- Patrones no lineales.
- El conocimiento del dominio lo pide (deuda/ingreso, nro. de ítems por compra, etc).

(Algunas) técnicas para crear features



- Matemáticas: A/B , $A*B$, $A-B$, etc.
- Agregados: promedios, totales por grupo, sumas, etc.
- Temporales: extraer día, hora, mes, de un timestamp.
- Texto: TF-IDF (importancia de una palabra), generación de embeddings.

Ejemplos comunes

- Fecha de ingreso → Antigüedad/categoría (senior, semi-senior, etc.) / tiempo desde el último evento (por ej promoción).
- Fecha de nacimiento → Edad.
- Timestamp → Hora del día / Es día hábil? / fin de semana? / feriado? / llovió ese día? / Temporada?
- Factores de riesgo (tabaquismo, hipertensión, diabetes, etc.) → Conteo / Nivel de riesgo ponderado / ocurrencia simultánea.
- Valores faltantes → Flag / conteo de faltantes por fila / faltante por dominio (ej: financiero, demográfico, etc.)
- Número de cel → localización geográfica a través del código de área.
- Sueldo/provincia/profesión → Salario promedio por provincia por profesión (ojo! media global si hay combinaciones muy raras)
- Transacciones/usuarios → número de transacciones por usuario.
- *Feedback* de un cliente → *Sentiment score* (positivo, neutral, negativo)
- Descripción de un producto → Palabras clave (descuento, oferta, premium, etc.)

Tip: conocer los modelos

- Agregar features con relaciones complejas (ej., IMC) cuando se trabaja con modelos lineales.
- Considerar agregar ratios, que son difíciles de aprender para casi todos los modelos.
- Los árboles pueden aprender casi cualquier combinación de features, pero cuando hay alguna combinación específica que nos interesa conviene crearla explícitamente (especialmente si hay pocos datos).
- Los features de tipo *aggregation* son especialmente útiles para modelos de tipo árbol porque estos no tienen la capacidad natural de combinar información de muchos features a la vez.

Cuidado con el data leakage



- Crear features después del split del dataset.
- Evitar features que son casi idénticos a la variable target o que son una consecuencia directa de esta y la "revelan". Ej: predecir transacciones fraudulentas usando la variable "se inició devolución del cargo".

Desbalance

Se produce cuando las clases (o etiquetas) que el modelo debe predecir no están representadas de manera proporcional en el dataset.

Causas:

- Sesgo al momento de recolectar los datos.
- Naturaleza del dominio en cuestión (causa más frecuente).

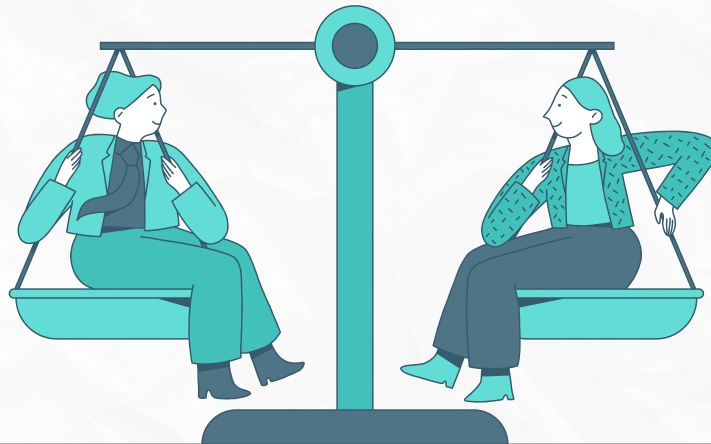


Desbalance

¿Por qué es un problema?

- Los modelos de ML tienden a aprender mejor de las clases más fuertes (mayoritarias) y tener un mal desempeño para las clases menos frecuentes (minoritarias).
- En consecuencia las métricas como precision, recall, F1-score y AUC-ROC, pueden ser engañosas.

Ejemplo: un modelo que siempre predice la clase mayoritaria tiene alta accuracy pero bajo recall en las clases minoritarias.



Técnicas para detectar desbalance

Técnica	Explicación
Análisis de la distribución	Análisis visual (gráfico de barras, pie chart) o cálculo de las proporciones de cada clase
Matriz de confusión	Según cómo se clasificaron las instancias puede revelarse un desbalanceo de clases.
Índice de Gini $G = 1 - \sum p_i^2$	Mide la pureza de los datos. Indirectamente puede reflejar si hay un desbalance entre clases. ~0.5 indica un dataset balanceado.
Entropía de Shannon $H = - \sum_{i=1}^n p_i \log_2(p_i)$	Mide la incertidumbre de los datos. Mientras más alta la entropía, mayor la incertidumbre, más balanceados los datos.

Técnicas para el tratamiento del desbalance

Técnica para balancear	Explicación
<i>Oversampling</i> (sobremuestreo)	Duplicar o recrear en forma sintética las muestras de la clase minoritaria: Ej. SMOTE (<i>Synthetic Minority Over-sampling Technique</i>) o IA generativa.
<i>Undersampling</i> (submuestreo)	Remover muestras de la clase mayoritaria.

Técnica para mitigar	Explicación
<i>Weighted Random Sampler</i> (muestreo aleatorio ponderado)	Utiliza muestreo con reemplazo proporcional a los pesos (generalmente inversos a la frecuencia de la clase). Esto simula un dataset equilibrado sin alterar los datos originales.
<i>Weighted Loss Functions</i> (Funciones de pérdida ponderadas)	Asigna pesos a las clases en la función de pérdida (ej. en <i>Cross-Entropy Loss</i>). Esto hace que el optimizador (ej. Adam) se enfoque más en minimizar errores en clases minoritarias.
Técnicas de ensamble	Algoritmos como <i>Balanced Random Forest</i> y AdaBoost tienen mecanismos internos para manejar el desbalance.

En desbalances extremos (1:1000), considerar recolectar más datos.

Importante!

Las técnicas de balanceo de clases solo se aplican a los datos de entrenamiento (validación y test deben reflejar el escenario real)



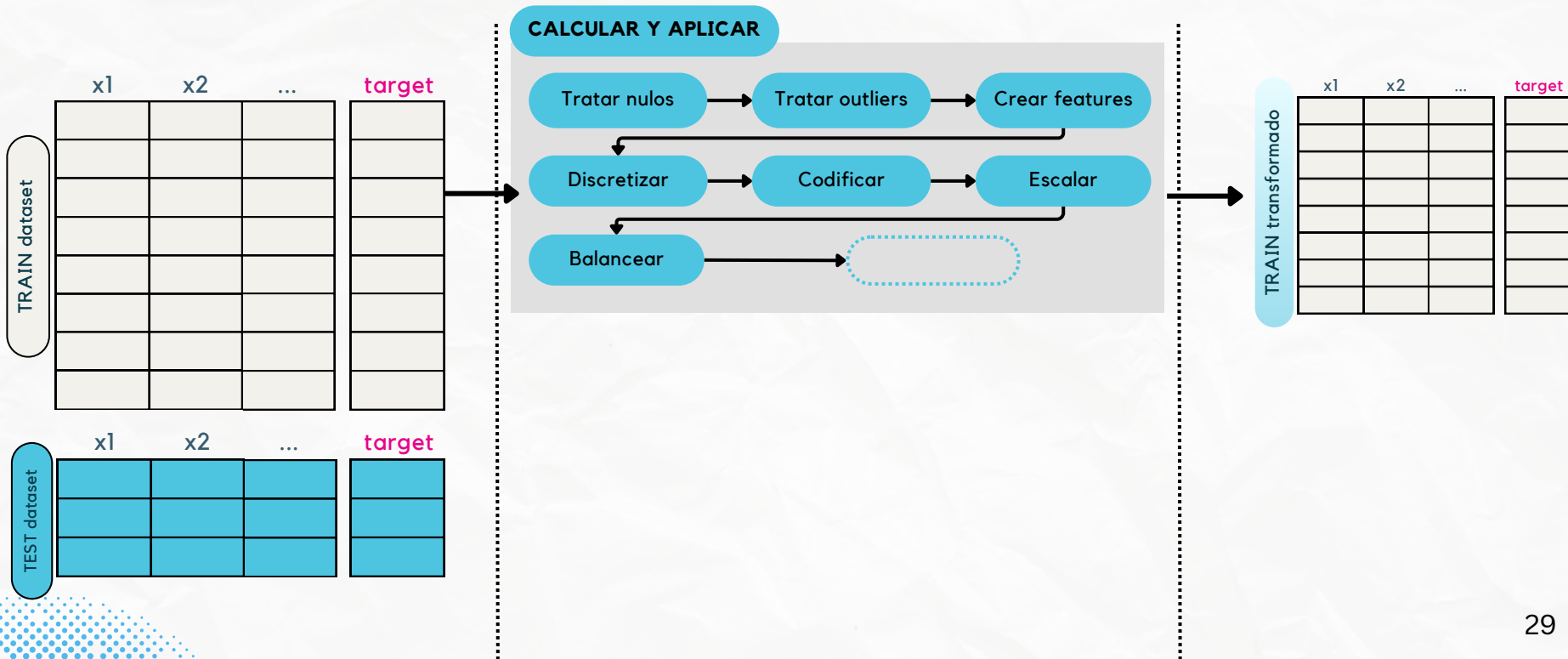
Ejemplo práctico



Flujo de preparación de datos

Split del dataset

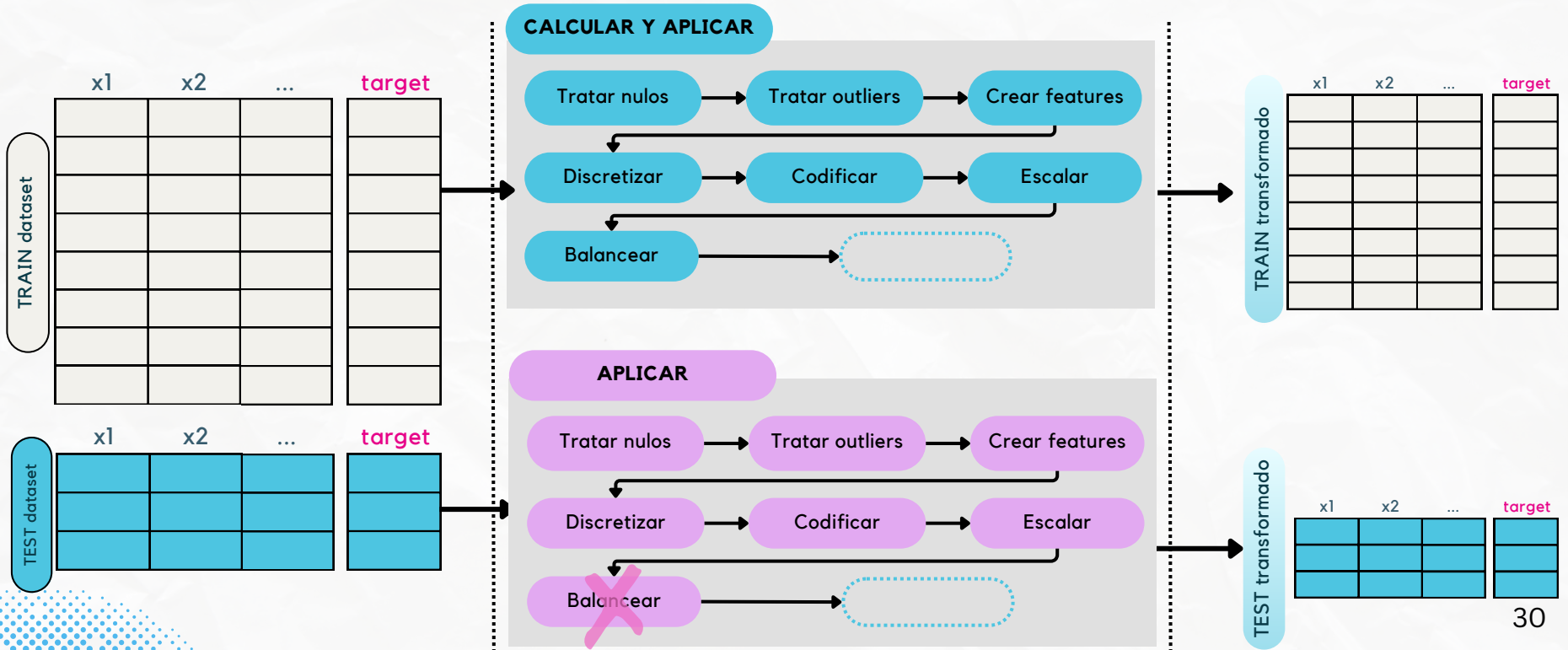
Calcular y aplicar las transformaciones



Flujo de preparación de datos

Split del dataset

Calcular y aplicar las transformaciones



Calcular y aplicar transformaciones

Transformación	Ejemplo	Calcular y aplicar en TRAIN	Aplicar en TEST
Tratar nulos	Imputar por la media	Calcular la media: 28 Imputar todos los nulos con 28.	Imputar todos los nulos con 28.
Tratar outliers	Clipping	Determinar outliers: > 65 Imputar todos los outliers con 65.	Imputar con 65 todos los valores mayores a 65.
Discretizar	Por rangos fijos	Calcular los rangos en base a la distribución de la variable: 10-20, 21,-30 Asignar la categoría: A (si el valor está entre 10 y 20) B (si está entre 21 y 30)	Asignar la categoría: A (si el valor está entre 10 y 20) B (si está entre 21 y 30)
Codificar	One hot encoding	Calcular las categorías: rojo, azul, verde. Crear tres columnas: R, A, V. Codificar rojo como 100, azul como 010 y verde como 001.	Codificar rojo como 100, azul como 010 y verde como 001.
Escalar	Normalizar	Calcular el máximo y el mínimo: 30 y 80. Calcular x' como $x - 30 / 50$.	Calcular x' como $x - 30 / 50$.

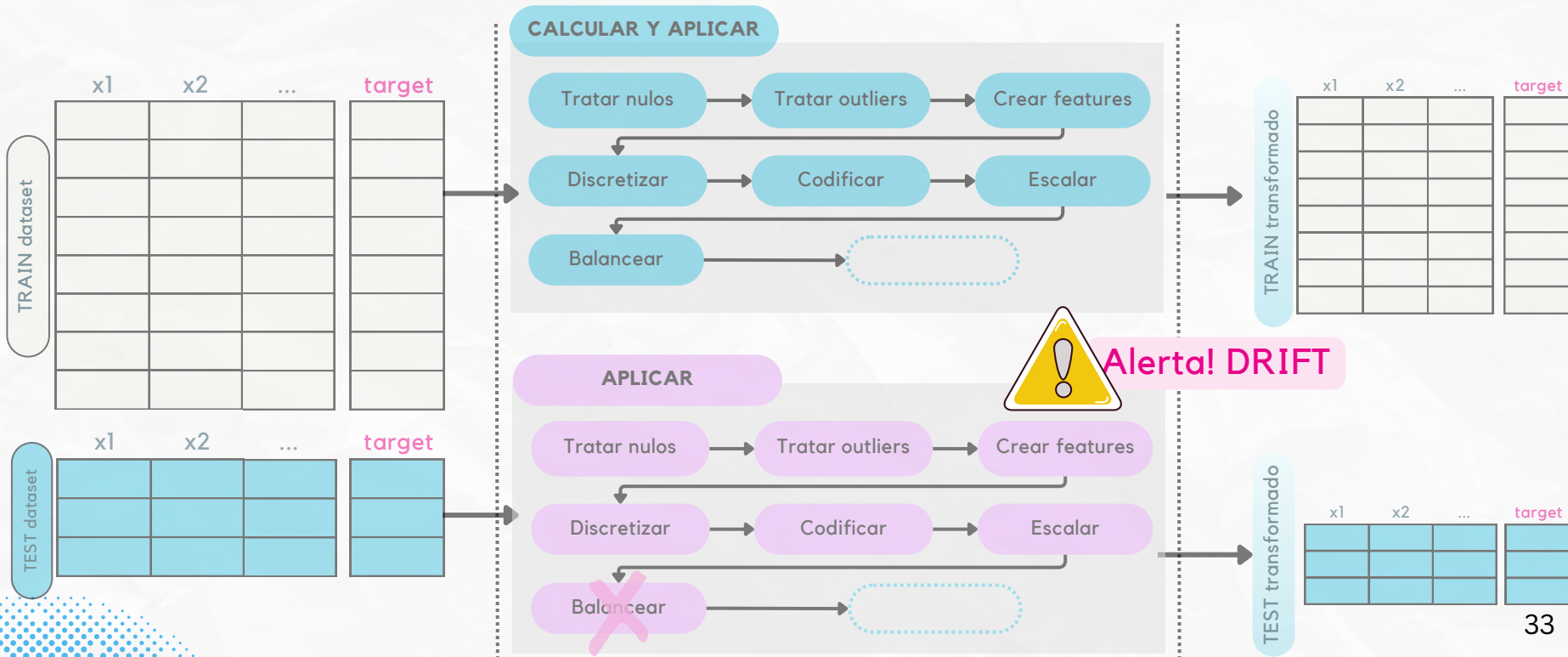
Calcular y aplicar transformaciones

Transformación	Ejemplo	Calcular y aplicar en TRAIN	Aplicar en TEST
Crear features (el cálculo no depende de otras observaciones)	IMC	$IMC = peso / (altura)^2$	$IMC = peso / (altura)^2$
Crear features (el cálculo depende de otras observaciones)	edad promedio por provincia	Calcular los valores de edad promedio por provincia. Ejemplo: Neuquén: 45. A las observaciones de Neuquén se les asigna 45.	Se asignan los valores según el cálculo hecho en entrenamiento. A las observaciones de Neuquén se les asigna 45.
Balancear	Oversampling	Aplicar sobremuestreo	-

Flujo de preparación de datos

Split del dataset

Calcular y aplicar las transformaciones





Data drift. Ejemplo 1: codificación.

Split del dataset

Calcular y aplicar las transformaciones

TRAIN dataset

A	B	C
	PERRO	
	PERRO	
	GATO	
	PERRO	

Identifico codificación:
OHE, dos categorías,
B_PERRO y B_GATO

Aplicar codificación

TRAIN transformado

A	B_PERRO	B_GATO	C
	1	0	
	1	0	
	0	1	
	1	0	

TEST dataset

A	B	C
	PERRO	
	GATO	
	PATO	

Aplicar codificación



Posibles soluciones:

- Ignorar la categoría desconocida.
- Re-analizar los datos/el problema.

TEST transformado

A	B_PERRO	B_GATO	C
	1	0	
	0	1	
	0	0	

Para una determinada variable, en el dataset de Test hay una categoría que no está en el dataset de Train.



Data drift. Ejemplo 2: escalamiento.

Split del dataset

Calcular y aplicar las transformaciones

TRAIN dataset	A	B	C
		10	
		20	
		5	
		50	

Normalización Min-Max:
Min = 5, Máx = 50
→ $(x - 5)/(50 - 5)$

Aplicar normalización

TRAIN transformado	A	B	C
		0,11	
		0,33	
		0	
		1	

TEST dataset	A	B	C
		15	
		2	
		55	

Aplicar normalización



Posibles soluciones:

- En la mayoría de los casos, esto no es un problema.
- Se puede hacer un *clipping* (forzar los valores a que caigan dentro del intervalo).

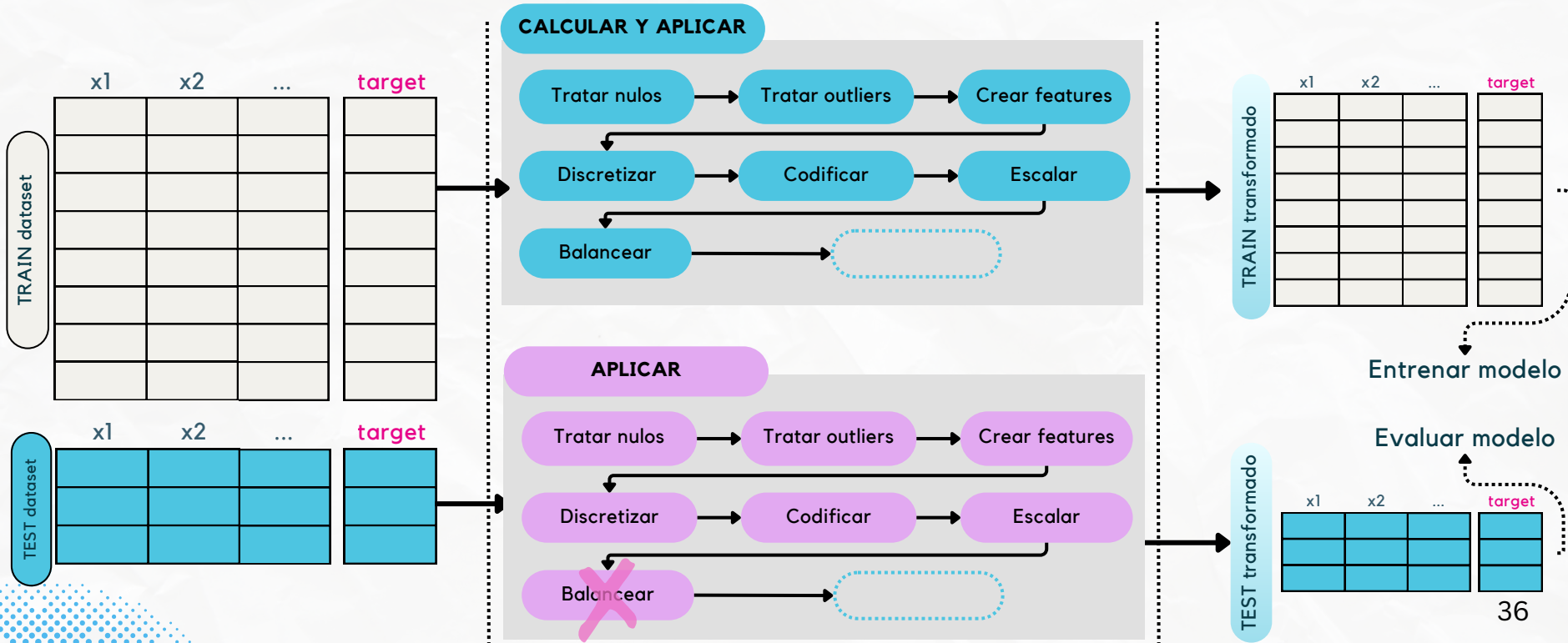
TEST transformado	A	B	C
		0,22	
		-0,06	
		1,11	

En Test una cierta variable toma un valor más grande que el máximo (y/o más chico que el mínimo) de esa variable en Train.

Flujo de preparación de datos

Split del dataset

Calcular y aplicar las transformaciones



¿Te quedaron dudas?



- Recordá que podés consultarnos durante la clase preniendo la cámara y hablando, o simplemente escribiendo en el chat.
- También podés escribirnos por correo las veces que sea necesario (por favor incluir a ambas docentes).



Esp. Lic. María Carina Roldán
macroldan@fi.uba.ar



Esp. Ing. Ariadna Garmendia
arigarmendia@gmail.com

Y si tenés un feedback, un pedido, una sugerencia, etc. y no te animás a decirlo, podés dejarlo por escrito en la encuesta voluntaria y anónima (disponible durante todo el bimestre).